Qwen

From Narrative Generator to File Compiler: A Governance Framework for Deterministic AI-Chat Pre-Wiring in HorrorPlace

Foundational Pillar I: Deterministic File Placement via Manifest and Spine-Based Routing

The primary objective of the proposed research framework is to transform the AI-Chat system from a free-form narrative generator into a deterministic "file compiler" within the HorrorPlace-Dead-Ledger-Network ecosystem . The cornerstone of this transformation is the establishment of a firm, non-negotiable foundation for file placement. The user's directive to prioritize deterministic file placement before considering other features like placeholders underscores a core principle: predictability, debuggability, and auditability are paramount . Without a guaranteed mapping from a given intent to a single, correct repository, path, schema, and tier, any subsequent logic for content generation or placeholder management becomes a source of ambiguity rather than utility. A change in a GitHub diff that shows a file moving between repositories or altering its fundamental structure would be chaotic and impossible to reason about; conversely, a diff showing only the filling of placeholders within a fixed structural envelope is clean, clear, and manageable. This approach directly aligns with established software engineering principles, particularly the "contract-first" methodology, where the rules of engagement are defined upfront and all subsequent actions must conform to them [4] [17]. The CHATDIRECTOR v1 architecture already embodies this by establishing manifests, spines, and envelopes as the single source of truth for routing, constraints, and structure . The task of this research framework is to tightly integrate the AI-Chat agent into this existing contract-first system, making it a compliant participant rather than an external entity operating on its own unenforced rules.

The mechanism for achieving this determinism lies in a sophisticated orchestration of three key architectural components: the manifest layer, the spine index, and the CLI orchestrator command (`plan`). Each plays a distinct yet interconnected role in resolving a request to a unique destination before any substantive content is generated. The

manifest layer, housed in files like `repo-manifest.hpc..json`, serves as the primary decision-making interface . These manifests contain explicit, machine-readable rules that map combinations of `objectKind` and `tier` to specific destinations. For each route, they define the target repository, the canonical path pattern for the artifact, the allowed schema whitelist, and crucial policy flags such as `OneFilePerRequest` and `RequireDeadLedgerRef` . To support AI-specific workflows, these manifests can be further enriched with explicit `aiAuthoringRules` and `authoringHints` sections. These could codify policies such as forbidding raw narrative text in Tier-1 repositories, mandating the inclusion of a `deadledgerRef` placeholder in Dead-Ledger-compatible repos, or redirecting artifacts with low RWF scores to designated quarantine paths . This provides a direct, auditable way to guide the AI's placement decisions based on organizational policy.

Complementing the manifests is the spine index, which acts as the definitive catalog of all permissible quantitative constraints within the system . It contains the eleven officially tracked invariants (CIC, MDI, AOS, RRM, FCF, LSG, HVF, DET, SPR, SHCI, RWF) and five entertainment metrics (UEC, EMD, STCI, CDL, ARR), along with their tier-dependent numeric ranges and cross-metric interactions . The spine is the ultimate arbiter of what is "legal" in terms of quantitative targets. During pre-wiring, the system must identify which invariants and metrics are always required for a given object kind and tier, such as `CIC`, `AOS`, `LSG`, `DET`, `SHCI`, and all five entertainment metrics . Furthermore, the spine should encode "safe defaults" for these values, allowing the `plan` step to propose valid targets even before the AI generates any content. Cross-metric constraints, such as "high DET raises the CDL floor" or "high CIC widens the SHCI band," must also be expressed in a shared semantic rules JSON format, ensuring that both the CLI and the AI operate under the same expectations . This prevents the AI from inventing arbitrary numbers and ensures that its choices are always bounded by the established governance model.

The orchestrator that brings these two layers together is the `plan` subcommand of the CHATDIRECTOR CLI tooling . This command is the central nervous system of the pre-wiring process. Its function is to take raw user intent—whether expressed as natural language (e.g., "Tier1 liminal mood for marsh region") or a partially filled JSON request—and normalize it into a fully populated, structured `chatDirectorRequestv1` envelope . In doing so, `plan` performs several critical functions. First, it resolves the abstract user intent into concrete metadata: the specific `objectKind`, `tier`, and corresponding `schemaref`. Second, it consults the spine to populate the request with default invariant and metric bands derived from the resolved `objectKind` and `tier`. Third, it may embed a pointer to a `chatDirectorSkeletonProfilev1` that provides default values for structural fields, effectively binding the request to a predefined archetype . Finally, the `plan` command enriches the request with a `generationGuide`,

a piece of metadata that provides the AI with essential context for its generation task. The research plan specifies extending `plan` to always output this fully normalized request, making pre-wiring an explicit and unavoidable step in the workflow .

To further constrain the AI's ability to invent invalid paths, the concept of a dedicated "routing-spine" index can be introduced or formalized within the existing spine index . This specialized index would map `objectKind` and `tier` pairs to canonical file patterns, such as `docs/<em>.md`, `schemas/tooling/</em>.json`, or `engine/*.lua`. Before emitting any content, the AI-Chat orchestrator can query this index to confirm the correct path pattern for its intended output, reinforcing the principle of "look up in spine/manifests and echo the plan" instead of "invent a path in the prompt" . When the `plan` command completes its work, it returns a complete `chatDirectorRequestv1` envelope containing all the necessary information for the AI to proceed. This envelope is no longer a vague prompt but a precise instruction set. It has already answered the fundamental questions of "where does this thing live?", "what is its legal identity (ID, invariants)?", and "what are the ground rules for its structure?". By locking down this placement and identity information in a pre-flight phase, the framework elevates AI-Chat from a creative writer to a disciplined compiler, transforming the entire authoring process into a repeatable, auditable, and governed pipeline. Every artifact produced is born with a fixed address and a clear lineage, ensuring that its journey through the constellation is predictable and verifiable.

Architectural Component	Role in Deterministic Routing	Key Artifacts & Rules
Manifest Layer	Primary decision engine for physical location and high-level policy enforcement.	`repo-manifest.hpc..json`: Routes `objectKind+tier` to `targetRepo`, `pathPattern`, `schemaWhitelist`. Policy flags include `OneFilePerRequest`, `RequireDeadLedgerRef`. Can be extended with `aiAuthoringRules` and `authoringHints`.
Spine Index	Definitive catalog of all legal invariants, metrics, and their permissible bands across tiers. Acts as the ultimate authority on quantitative constraints.	`invariants-spine`, `entertainment-metrics-spine`: Defines valid ranges, tier overrides, and cross-metric interactions for all governance variables.
CLI `plan` Command	Orchestrator that normalizes intent and resolves all routing and constraint information into a single, structured request envelope.	Accepts natural language or partial JSON. Outputs a fully populated `chatDirectorRequestv1` with resolved `objectKind`, `tier`, `schemaref`, and a `generationGuide`.
Routing-Spine Index	Optional, specialized layer providing canonical file path patterns for different `objectKind` and `tier` combinations to prevent path invention.	Maps `objectKind+tier` to canonical patterns (e.g., `engine/<em>.lua`, `contracts/</em>.json`). Used by the orchestrator to validate path construction.

This layered, rule-based approach ensures that every AI-generated file is not just a blob of content, but a well-documented, traceable asset from its inception. The

`chatDirectorRequestv1` becomes the single source of truth for the artifact's intended properties, and the pre-wiring phase guarantees that this truth is grounded in the authoritative sources of the `repo-manifest` and the `spine`. This foundational pillar of determinism is the prerequisite for all other advanced features, creating a stable base upon which more complex behaviors like placeholder management and multi-phase generation can be built reliably.

Foundational Pillar II: Contract-First Component Wiring with Placeholder-Aware Scaffolding

Once deterministic file placement is secured, the next foundational pillar focuses on the internal structure of the artifact itself. The user's preference for a "contract-first ID wiring with placeholders" establishes a powerful abstraction for how the AI-Chat system should interact with the `HorrorPlace-Dead-Ledger-Network` . Instead of generating complete files with prose and code from the outset, the AI's initial and primary task is to construct the referential graph of the system—the network of relationships and dependencies between entities. This approach treats the AI as a tool for generating IDs (`regionId`, `seedId`, `historyBundleId`), hooks (`luaModule`, `H.Node` IDs), and other structural metadata. These elements establish the connectivity and logical integrity of the final artifact set, forming a validated scaffold upon which content can later be hung. This method directly supports the goal of separating graph wiring from content filling, creating a clearer separation of concerns that simplifies debugging, enhances CI/CD processes, and makes GitHub diffs far more meaningful. A diff focused on structural changes (adding a new dependency, changing a hook) can be reviewed independently of a diff focused on prose or code changes.

This structural approach is enabled by the "contract-first" paradigm, where the schema defines the relationships before the content [4] [17]. The `chatdirector-envelope-v1.json` schema already enforces this discipline with `additionalProperties: false`, preventing the introduction of arbitrary fields . The research extends this principle to the composition of multiple files. An AI-Chat session, guided by the `generationGuide` from the `plan` command, can be tasked with generating a family of related artifacts. For example, it might first generate a `regionContractCard` that declares its own `regionId`, then use that ID to generate a `seedContractCard` that lists the `regionId` as its parent, and finally generate a Lua module stub that references both IDs. The validation pipeline would then check the referential integrity of this entire set, ensuring that every ID used actually points to a valid, existing artifact. This is

analogous to building a database schema with foreign keys before populating the tables, where the structural validity is enforced before any data is inserted [31]. The CHATDIRECTOR v1 lifecycle spec, with its phases (`Schema0`, `Registry1`, `Bundles2`, etc.), provides a natural framework for this process, as each phase represents a stage of increasing structural complexity .

To make this scaffolding process effective, the system leverages `chatDirectorSkeletonProfilev1`, which maps an `archetypeId` plus `objectKind` and `tier` to a default envelope and partial contract body . These skeletons provide the necessary scaffolding for the AI, offering predefined structural defaults. For instance, a skeleton for a `moodContract` in a Tier-1 lab environment might come with safe-default invariant bands from the spine, placeholder IDs, and empty-but-present content fields. The `plan` command can either embed a skeleton directly into the `chatDirectorRequestv1` or return a pointer to a specific profile for the AI to honor . This ensures that the AI starts from a position of structural soundness. The research should define whether the AI is allowed to modify these skeleton-provided numeric targets (invariants/metrics) or if it is restricted to refining the content while keeping the structural targets fixed. Telemetry hooks should be established to compare the outcomes of skeleton-based generations versus "from scratch" attempts across various metrics like UEC, ARR, and validation success rates, providing data-driven guidance on when and how to adjust skeletons .

The third critical element of this pillar is the design of placeholder-aware schemas. Placeholders are not merely informal notes like `"TODO: write trauma description"`; they are first-class citizens in the schema, designed to be validated and managed by the system . The framework must introduce explicit, constrained mechanisms for marking content as incomplete. One approach is to add a dedicated field to the envelope or contract schemas, such as `contentStatus` with an enum value of `"placeholder"|"draft"|"final"` . Another is to use a boolean flag like `isPlaceholder: true` on specific fields or blocks of content . This allows the CI pipeline and other tooling to treat placeholder artifacts differently—for example, by applying stricter rules, preventing them from being promoted to higher tiers, or flagging them for review. This transforms placeholders from simple comments into a structured mechanism for managing partial states.

Furthermore, placeholder logic should be codified using explicit enums or tokens rather than freeform text. Instead of leaving a field with an arbitrary string, the schema can require a specific value, such as `"PLACEHOLDER_RESEARCH_COPY"`, `"PLACEHOLDER_EXAMPLE_ONLY"`, or `"REDACTED_FOR_TIER1"` . This allows validation to mechanically detect and police the presence of placeholders, ensuring they

are used appropriately and do not leak into production-ready artifacts. This technique is supported by the capabilities of modern JSON Schema validators to enforce `enum` restrictions [7] [15]. Placeholders should only be allowed in designated, non-critical fields. For example, a `seedContractCard` schema might have a `narrativeBlurb` field for human consumption and a separate `placeholderNarrative` field for AI use, while core structural fields like `seedId`, `variantTags`, and invariant bands must always contain real, validated data . This keeps the structure and governance of the artifact intact even when some surface-level content is left for later completion. By combining contract-first ID wiring with a robust, schema-enforced placeholder system, the framework enables the AI to generate structurally complete graphs that are ready for validation and routing, while safely deferring substantive content creation until a later phase. This creates a hybrid generation model that is both powerful and disciplined.

Feature	Description	Implementation Strategy	Governing Component
Contract-First ID Wiring	AI generates structural metadata (IDs, hooks) before content. Establishes referential integrity.	Use schemas that enforce the generation of `regionId`, `seedId`, `deadledgerRef`, `luaModule`, etc., as required fields.	`chatdirector-envelope-v1.json`, Family-specific schemas (`chatdirector-moodcontract-v1.json`, etc.)
Scaffolding via Skeletons	Provides predefined structural defaults to guide the AI and ensure a baseline level of correctness.	Define `chatDirectorSkeletonProfilev1` mapping `archetypeId+objectKind+tier` to default envelopes and partial bodies.	`plan` command embeds or points to a skeleton in the `chatDirectorRequestv1`
Explicit Status Markers	Introduces a dedicated field to indicate the completeness of content.	Add a `contentStatus` field to schemas with an enum: `"placeholder"|"draft"|"final"`.	JSON Schema definition for envelopes and contracts.
Structured Placeholder Enums	Replaces informal placeholder text with a controlled set of valid tokens.	Define `enum` fields for sensitive or optional content, e.g., `["PLACEHOLDER_EXAMPLE_ONLY", "REDACTED_FOR_TIER1", ""]`.	JSON Schema `enum` keyword on specific fields [7] [15].
Designated Placeholder Fields	Restricts placeholders to specific, non-critical fields to preserve core structural integrity.	Design schemas to have separate, clearly named fields for AI placeholders (e.g., `placeholderNarrative`) vs. human-facing content (e.g., `flavorText`).	Family-specific contract schemas (`chatdirector-moodcontract-v1.json`, etc.)

This second pillar solidifies the AI's role as a builder of systems, not just a writer of documents. By focusing on the graph of connections first and using a structured, validated approach to handling incomplete content, the framework ensures that every artifact produced is not only correctly placed but also structurally sound and compliant with the governing contracts of the `HorrorPlace-Dead-Ledger-Network`.

Foundational Pillar III: Enforcing Strict Phase Separation and Validation-Driven Revision

The third and final foundational pillar of the research framework introduces a disciplined, multi-phase workflow that governs the entire authoring process, enforcing rigor and reliability. This pillar is built upon the principle of strict phase separation: a clear distinction between a graph-wringing phase, where the artifact's structure and relationships are defined and validated, and a content-filling phase, where substantive prose, dialogue, or code is generated within the rigid constraints of the pre-validated graph. This separation is not merely a conceptual exercise; it is a practical implementation of a defensive programming strategy that yields significant benefits in terms of debuggability, CI reliability, and the overall predictability of AI behavior. The user's explicit preference for this stricter separation highlights a desire to move away from a "unified generation approach" and toward a more systematic, verifiable process .

The workflow begins with Phase 1: Graph Wringing and Validation. In this initial stage, the AI-Chat orchestrator is tasked with generating a structurally complete artifact. This means producing a file that contains all required structural fields—valid IDs, hooks, invariant bands, metrics, and Dead-Ledger surface declarations—but may leave substantive content fields as empty strings or explicit placeholder markers . The moment this skeletal artifact is generated, it is immediately passed to the four-layer CHATDIRECTOR v1 validation pipeline: Schema, Invariant, Manifest, and Envelope, executed in that fixed order . This immediate feedback loop is the engine of determinism. If the artifact fails validation at any layer, the system receives a structured diagnostic report encapsulated in a `chatDirectorErrorv1` schema . Crucially, this error object must contain machine-friendly details for every failure, including a `code`, the `layer` of failure (e.g., `Invariant`), a `jsonPointer` indicating the exact field in error, the `expected` value(s), and a `remediation` suggestion for the AI to correct its mistake .

This validation-driven error loop is the most critical part of the framework. When validation fails, the AI-Chat is not simply told "your file is wrong." It is presented with a precise, actionable report. For example, the diagnostics might state: "ERR_INVARIANT_OUT_OF_BOUNDS at `/invariants/DET` (value: 85). Expected: integer in range [50, 75]. Please reduce the DET value." Armed with this specific feedback, the AI can attempt a second pass at generation, correcting only the specified errors. This transforms the AI from a black box that produces unpredictable outputs into a responsive system that learns from its mistakes within the strict confines of the pre-defined contract. This iterative revision process continues until the artifact passes all validation checks. Only after successful validation does the system proceed to Phase 2:

Content Filling. At this point, the AI's role shifts from one of structural construction to one of content enrichment. It is now permitted to replace the empty or placeholder-filled content fields with prose, dialogue, or code. However, it operates under the constraint that it cannot alter the validated structural elements. Its job is to fill in the blanks of a blueprint that has already been approved by the governance system. This clear division of labor ensures that the integrity of the artifact's structure is never compromised during the content creation phase.

This two-phase model directly addresses the user's goals for improved CI reliability, clear GitHub diffs, and debuggable AI behavior. Because the structural and content-generating steps are separated, a CI pipeline can run different, targeted checks on each. For a PR containing a Phase 1 commit, the focus is on schema compliance, invariant ranges, and referential integrity. For a Phase 2 commit, the focus might be on code style, prose quality, or runtime metrics. This makes reviews more efficient and targeted. A GitHub diff for a Phase 1 change will show only modifications to IDs, metrics, and other structural fields, making it easy to verify that the artifact's wiring is correct. A diff for a Phase 2 change will show only the filling of content placeholders, making it easy to assess the quality of the narrative or code without being distracted by structural noise. This clarity is invaluable for debugging. If an issue arises, developers can isolate whether the problem is in the artifact's structure (a "wiring" error) or its content, and the AI's revision loop ensures that "fix wiring errors first, then retry content" becomes a standard, automated troubleshooting procedure . The promotion predicates defined in the lifecycle spec, which are encoded as machine-readable rules, also become more effective in this model. Since every artifact entering the content-filling phase is already structurally sound and has passed its initial validation gates, the promotion criteria (e.g., coverage thresholds, SHCI bands, fully resolved `referencedIds`) are more likely to be met, and the CI process can confidently promote the artifact to the next phase of the lifecycle . This disciplined, validation-centric workflow is the keystone of the entire framework, ensuring that every piece of AI-generated content that lands in the repository is not only compliant but also predictable and maintainable.

Phase	Objective	AI Responsibilities	Validation Focus	Outcome
Phase 1: Graph Wringing	Generate a structurally complete, referentially intact artifact.	Generate all required structural fields: valid IDs, hooks, invariant/metric bands, and Dead-Ledger surfaces. Leave content fields as empty or placeholder markers.	Schema, Invariant, Manifest, Envelope layers. Check for structural correctness, valid IDs, and adherence to numeric bands.	A validated `chatDirectorRequestv1` envelope with a structurally sound but content-empty artifact.
Phase 2: Content Filling	Fill placeholder and empty content fields with prose, dialogue, or code.	Replace placeholder markers with substantive content. Respect the underlying structure from Phase 1. Do not alter IDs, hooks, or invariant bands.	Code style linters, prose quality checks, or other post-structural validation rules.	A fully populated, finalized artifact ready for final approval and Dead-Ledger attestation.
Revision Loop	Correct errors identified during validation.	Receive a `chatDirectorErrorv1` with `jsonPointer`, `expected`, and `remediation`. Attempt a corrected generation of either Phase 1 or Phase 2.	Re-run the specific validation layer that failed.	Iteratively refine the artifact until it passes all validation checks for its current phase.

By enforcing this strict, multi-stage process, the framework embeds governance directly into the authoring workflow. It moves beyond simply reacting to errors found in CI and instead proactively constrains the AI's behavior at every step, ensuring that only high-quality, compliant artifacts progress through the system.

Advanced Governance: Proactive Controls Beyond Core Pre-Wiring

While the three foundational pillars establish a robust framework for deterministic file placement, contract-first wiring, and phase-separated generation, the system's maturity and resilience depend on the addition of more advanced, proactive governance controls. These enhancements move the system from a static set of rules to a dynamic, learning, and deeply integrated governance apparatus that anticipates potential issues before they arise. They build upon the core CHATDIRECTOR v1 architecture but extend it to address the specific challenges of an AI-driven authoring workflow, such as stale data assumptions, role-based permissions, and the need for self-constraint by the AI itself. These advanced controls are not mutually exclusive with the core pillars; rather, they act as force multipliers, making the entire system more secure, reliable, and intelligent.

The first and most critical enhancement is the introduction of a `discovery-contract.v1` as a hard precondition for any authoring activity . Before an AI-Chat session can even begin processing a `chatDirectorRequestv1`, it must first present a

discovery contract. This small, strict schema records exactly which spine documents, repo manifests, registries, and capability catalogs the AI session has loaded into its context . CHATDIRECTOR would refuse any request that does not reference a fresh, valid discovery contract. This measure is designed to prevent "stale data" errors, where an AI might inadvertently operate on outdated routing rules or invariant bands because its context was not synchronized with the latest changes in the constellation. The implementation would involve defining Rust and Lua helpers to validate the freshness of the discovery contract (e.g., checking against a known-good spine version hash), wiring this validation into the `plan` command so that request normalization always checks the discovery contract first, and adding CI tests that prove no authoring flow can bypass this mandatory gate . This forces the AI to engage with the governance system at the very beginning of its workflow, grounding its operations in the single source of truth.

Complementing the discovery contract is the concept of a read-only capability catalog. Derived from the schema spine and manifests, this catalog exposes a prompt-safe JSON view of all currently allowed operations within the session's scope . It would describe available `objectKinds`, permissible `phases` and `tiers`, legal invariants and metrics, and the associated routing rules. The research should specify how this catalog is serialized for consumption by the AI in languages like Rust (`ChatDirector.catalog`) and Lua (`H.Discovery.capabilityCatalog`) . The objective is to empower the AI to query this catalog and self-constrain *before* it proposes any artifact. Instead of guessing what is legal, the AI can ask, "What `objectKind`s are available for Tier 2?" or "What are the valid ranges for `SPR` in this context?". This transforms the AI from a passive recipient of constraints into an active participant in its own governance. Key tasks for this objective include defining the catalog schema, clarifying which invariant and metric descriptions are safe to include in prompts without leaking sensitive operational details, and designing telemetry to measure how catalog usage correlates with improvements in validation success rates and reductions in misrouting incidents . This is akin to providing an AI with a comprehensive dictionary and grammar guide before asking it to write a novel.

A third major advancement is the shift from a single global contract surface to per-agent `ai-safe-authoring-profile-v1` cards. This adds a crucial layer of security and fine-grained, role-based access control . Different AI agents will have different responsibilities and levels of trust. An "Architect" agent profile might have broader permissions, allowing it to operate across more tiers and modify a wider range of invariants, whereas an "Implementer" profile might be restricted to lower tiers and specific object kinds. These profiles would encode numeric caps and floors on invariants and metrics, as well as explicit lists of allowed repositories. The CHATDIRECTOR system would read these profiles, intersect their constraints with the spine-defined bands, and use the resulting, agent-specific bounds during the `plan` phase to produce a tailored

`generationGuide` for that particular agent . This research needs to determine how flags like `allowsClamp` (whether the system can automatically adjust a value outside the profile's bounds to the nearest allowed limit) and `allowsSchemaUpgrade` (whether an agent can request a newer, more permissive schema version) interact with the pre-wiring and promotion rules. Studying the impact of different profile settings on CI health and Dead-Ledger eligibility over time will be crucial for tuning these roles effectively .

Finally, to support a distributed authoring ecosystem, the same rigorous pre-wiring logic must be made available to tools beyond chat interfaces. This involves defining minimal contracts and protocols for IDE/editor plugins (for Unreal, Godot, VS Code) that call the CHATDIRECTOR `plan` and `validate-response` commands on behalf of users . This objective covers how editor context—such as the open file, current selection, and repository root—is normalized into a `chatDirectorRequestv1`. It also involves designing how pre-wiring hints from the `RouteExplanation` and diagnostics from validation can be fed back into the editor's UI to provide real-time feedback to the developer. The overarching goal is to ensure that pre-wiring logic is identical across chat integrations and IDEs, using the same discovery contracts, capability catalogs, manifests, and agent profiles. This creates a single, unified governance surface for all forms of authoring, ensuring consistency and preventing developers from accidentally circumventing the rules by using different tools. Together, these advanced governance controls transform the system from a simple set of rules into a proactive, adaptive, and deeply integrated security and quality assurance platform.

System Evolution and Feedback Loops for Continuous Improvement

A static set of rules, however well-designed, is insufficient for a dynamic system driven by generative AI. The performance of the AI-Chat agent, the appropriateness of the spine's invariant bands, and the effectiveness of the skeleton profiles are not constants; they evolve over time as the AI models improve and the project's needs change. Therefore, a critical component of the research framework is the establishment of robust telemetry and feedback loops that allow the system to learn from its own operation and adapt accordingly. This moves the system from a purely prescriptive model ("you must follow these rules") to a descriptive-and-prescriptive model ("we see how you perform, and we will help you follow better rules"). This continuous improvement cycle is essential for maintaining the long-term health, efficiency, and accuracy of the AI pre-wiring framework.

The foundation of this evolutionary process is the extension of `GenerationMetrics` logging to capture detailed telemetry at every stage of the AI authoring workflow . While the initial plan calls for logging basic pass/fail outcomes, this expanded telemetry strategy must track a much richer set of signals specific to pre-wiring behavior . The proposed telemetry schema should include fields to log:

- **Discovery-Contract Freshness:** Whether the `discovery-contract.v1` used was current and matched the expected spine version.
- **Capability Catalog Usage:** Which entries from the capability catalog were queried by the AI before generation.
- **Profile Violations:** Instances where the AI's initial proposal fell outside the bounds of its assigned `ai-safe-authoring-profile-v1`.
- **Routing Corrections:** Data on how often the `plan` command had to correct an AI's initial guess about the target repository or path.
- **Retry Counts Per Error Code:** A granular breakdown of how many times the AI attempted to regenerate an artifact for each specific validation error code returned by the `chatDirectorErrorv1` schema .

This rich dataset, aggregated by `objectKind`, `tier`, and `repo`, provides deep insights into the AI's performance and the quality of the pre-wiring guidance it receives. For instance, analyzing this telemetry can reveal which skeleton profiles systematically lead to validation failures, suggesting they need their default values adjusted. It can identify spine bands that are too tight for the AI to satisfy without excessive retries, signaling a need for relaxation. It can highlight tiers or object kinds where AI performance is consistently lower, indicating a need for more detailed guidance or different skeleton archetypes .

These findings are then fed back into the system's core components through a semi-automated tuning process. The analysis of telemetry data should drive updates to the spine safe defaults, the `authoringHints` within repo manifests, the numeric bounds within `ai-safe-authoring-profiles`, and the content of the `generationGuide` itself . For example, if data shows that the AI frequently violates the `SPR` invariant band, the `generationGuide` could be updated to provide a more prominent warning or a narrower suggested range. If a certain skeleton profile is shown to correlate with higher validation success rates, it could be made the default for a specific archetype. This turns the system from a static specification into a living, evolving entity that improves continuously based on observed performance data rather than static design assumptions [13] . This closed-loop refinement is a hallmark of mature AI systems, where the model's interaction with its environment informs its future behavior and the design of its operating environment.

To complement this data-driven evolution, the framework must incorporate a strong safety net against regressions. This is achieved by creating an AI failure regression corpus and a corresponding golden test harness focused specifically on pre-wiring behaviors . This corpus would consist of a small family of `.json` test cases derived from real-world AI-Chat errors encountered during development. Each test case would capture the original `chatDirectorRequestv1`, the full diagnostics output from the validation pipeline, and a corrected payload that successfully passes validation . These "golden cases" serve as immutable benchmarks for the pre-wiring logic. By wiring them into the CI/CD pipeline, the team can ensure that future changes to the `plan` command, spine definitions, repo manifests, or agent profiles cannot silently break established behaviors. Any change that causes one of these regression tests to fail would be caught immediately, preserving the stability and reliability of the framework. This practice, common in robust software development, is especially critical in a system as complex as this, where subtle changes in routing or validation logic can have widespread consequences. Together, the combination of detailed telemetry and a rigorous regression suite creates a powerful engine for both innovation and stability, ensuring the pre-wiring framework remains both cutting-edge and dependable.

Telemetry Signal	Purpose	How It Drives Improvement
Catalog Usage	Measures how often the AI consults the capability catalog before generating.	Correlate with validation success rates. Low usage might indicate the catalog is unclear or incomplete, prompting documentation or schema improvements.
Profile Violations	Tracks instances where the AI's proposal exceeds its agent profile's limits.	Identify overly restrictive profiles or proposals that are too aggressive. Inform adjustments to profile bounds or provide better guidance in the `generationGuide`.
Routing Corrections	Logs how often the `plan` command had to override the AI's initial routing guess.	High correction rates suggest the AI is poorly guided on routing rules. This can inform the creation of more detailed `authoringHints` in manifests or better `generationGuide` explanations.
Retry Counts Per Error Code	Provides a granular breakdown of regeneration attempts for each specific validation failure.	Pinpoint the most difficult validation rules for the AI to satisfy. This can trigger targeted improvements, such as refining the associated error message or adjusting the spine band for that specific invariant.
Validation Success/Failure	The primary outcome metric for each generation attempt.	Aggregate across `objectKind/tier` to identify systemic weaknesses in spine bands, skeleton profiles, or `generationGuide` quality.

This commitment to measurement and iteration ensures that the framework does not become brittle over time. It acknowledges that the optimal configuration for guiding an AI is not a one-time setup but an ongoing process of observation, analysis, and adjustment.

Synthesis and Strategic Implications for the Authoring Ecosystem

The research framework detailed in this report outlines a comprehensive strategy to transform the AI-Chat system from an unbounded creative tool into a deterministic, governance-compliant "file compiler" for the `HorrorPlace-Dead-Ledger-Network`. This synthesis reframes the core challenge: the goal is not merely to generate content, but to generate *correctly structured, traceable, and auditable assets* that fit seamlessly into a larger, complex ecosystem. The framework achieves this by implementing a three-pillar approach—deterministic routing, contract-first wiring, and strict phase separation—that collectively imposes a rigorous, machine-enforced structure on the AI's creative process. This structure is not a limitation but an enabler, providing the predictability and reliability necessary for large-scale, collaborative development involving both humans and artificial agents.

The strategic implication of this framework is profound. It positions AI-Chat not as an autonomous creator, but as a highly skilled assistant operating under a well-defined contract. The entire process is reimagined around the metaphor of a compiler: the `chatDirectorRequestv1` is the source code, the multi-layer validation pipeline is the assembly and linking process, and the final generated file is the compiled binary. This reframing fundamentally alters the human-in-the-loop experience. The developer's role shifts from manual authorship to that of a "guidance engineer" or "system director." Their primary tasks become crafting better `generationGuides`, curating and improving skeleton profiles, defining more effective `aiAuthoringRules` in manifests, and interpreting validation diagnostics to provide corrective feedback. The AI handles the complex, rule-based aspects of file generation, freeing the human to focus on higher-level architectural and creative decisions.

Furthermore, the framework's emphasis on proactive governance, through mechanisms like discovery contracts, capability catalogs, and AI-safe authoring profiles, builds a system that is inherently more secure and less prone to accidental errors. By forcing the AI to declare its knowledge base and adhere to role-based permissions from the outset, the system mitigates risks associated with stale data, unauthorized operations, and inconsistent behavior. The integration of telemetry and a regression corpus ensures that the system is not static; it is a dynamic entity capable of learning from its performance and adapting its own rules to improve over time. This creates a virtuous cycle of improvement where the system's intelligence grows alongside the AI models it utilizes.

Ultimately, the successful implementation of this research framework will yield a `HorrorPlace-Dead-Ledger-Network` authoring ecosystem that is uniquely suited to its ambitious goals. It combines the creative power of generative AI with the structural integrity and governance of a traditional software development lifecycle. It produces artifacts that are not only compelling in content but also mechanically precise in form, ensuring they can be reliably integrated, version-controlled, and proven within the VM-constellation and Dead-Ledger model. This approach provides a scalable and sustainable blueprint for leveraging AI in complex, regulated environments, demonstrating that with the right framework, creativity and control are not opposing forces but complementary partners in the creation of digital worlds.

Reference

1. [PDF] Documentation for Confluence 5.5 https://product-downloads.atlassian.com/software/confluence/downloads/documentation/Confluence_5-5-0_AdminGuide_PDF_CONF55-050514-2301-28.pdf
2. Code Igniter for Rapid PHP Application Development - Academia.edu https://www.academia.edu/7100376/Code_Igniter_for_Rapid_PHP_Application_Development
3. Mat Plot Lib | PDF - Scribd https://www.scribd.com/doc/61445077/Mat-Plot-Lib
4. Contract-First Integration: A Practical Guide for Developers https://dev.to/wallaceespindola/contract-first-integration-a-practical-guide-for-developers-49nf
5. Repairable Contract-Based Skills for Multimodal Web Agents - arXiv https://arxiv.org/html/2603.20340v3
6. Power Query Documentation | PDF | Databases | Microsoft Excel https://www.scribd.com/document/495287948/Power-Query-documentation
7. How to perform JSON schema property validation based on enum ... https://stackoverflow.com/questions/61164815/how-to-perform-json-schema-property-validation-based-on-enum-value-in-another-pr
8. Best practices to add nuances to enums - Prompting https://community.openai.com/t/best-practices-to-add-nuances-to-enums/1355280
9. [PDF] Introduction to the System z Hardware Management Console https://www.redbooks.ibm.com/redbooks/pdfs/sg247748.pdf

10. Manage Suggested Values in the API | Adobe Experience Platform https://experienceleague.adobe.com/en/docs/experience-platform/xdm/tutorials/suggested-values
11. Fields - marshmallow 4.3.0 documentation https://marshmallow.readthedocs.io/en/stable/marshmallow.fields.html
12. Document Structure Overview | PDF | Internet Forum | Php - Scribd https://www.scribd.com/document/843445338/Raft-Small-Words-Lowercase
13. I worked at a semantic layer startup a few years ago. | Kenny Ning https://www.linkedin.com/posts/kenny-ning_i-worked-at-a-semantic-layer-startup-a-few-activity-7396180969491427329-28_K
14. A Survey on LLM-based Code Generation for Low-Resource ... - arXiv https://arxiv.org/html/2410.03981v3
15. Enum restrictions in strict JSON schemas for responses API https://community.openai.com/t/enum-restrictions-in-strict-json-schemas-for-responses-api/1366742
16. java - JSON Schema - Enum of Objects - Stack Overflow https://stackoverflow.com/questions/67626506/json-schema-enum-of-objects
17. Prompt Engineering is Dead: Shift to Contract-First AI with JSON ... https://www.linkedin.com/posts/discoverinme_visionarchitect-jsonschema-systemarchitecture-activity-7417166168462999565-tJOl
18. Annual Meeting of the Association for Computational Linguistics ... https://aclanthology.org/events/acl-2025/
19. How to GIWAXS: Grazing Incidence Wide Angle X − Ray Scattering ... https://advanced.onlinelibrary.wiley.com/doi/10.1002/aenm.202300760
20. [PDF] Deliverable D3.2 Title: Machine Learning and Automation for Crime ... https://ec.europa.eu/research/participants/documents/downloadPublic?documentIds=080166e5dd66c92b&appId=PPGMS
21. Chemistry beyond the scale of exact diagonalization on a quantum ... https://www.science.org/doi/10.1126/sciadv.adu9991
22. Decentralized digital health ecosystems: a unified architecture for AI ... https://www.frontiersin.org/journals/digital-health/articles/10.3389/fdgth.2025.1685628/full
23. Limitations on enum values in Structured Outputs - API https://community.openai.com/t/limitations-on-enum-values-in-structured-outputs/984464
24. make jsonSchema enum attribute conditionally required with useful ... https://stackoverflow.com/questions/60594124/make-jsonschema-enum-attribute-conditionally-required-with-useful-error-message

25. [PDF] Exploration of the peptidergic modulation of pain - HAL theses https://theses.hal.science/tel-04412707v1/file/CLAUSS--CREUSOT_Etienne_2023_ED414.pdf
26. [PDF] The proceedings of the 6th Intercontinental Geoinformation Days https://www.researchgate.net/profile/Aydin-Alptekin-2/publication/374544736_The_proceedings_of_the_6th_Intercontinental_Geoinformation_Days/links/652310e1fc5c2a0c3bc2cce0/The-proceedings-of-the-6th-Intercontinental-Geoinformation-Days.pdf
27. Articles Home | New Era Technology https://digital.neweratech.com/articles
28. Appl. Sci., Volume 12, Issue 8 (April-2 2022) – 390 articles - MDPI https://www.mdpi.com/2076-3417/12/8
29. Umbraco .NET Core Development Guide | PDF - Scribd https://www.scribd.com/document/899753640/Jones-J-Umbraco-NET-Core-Mastery-Using-Umbraco-10-or-9-2022
30. [PDF] Developing Applications with Oracle JDeveloper https://docs.oracle.com/en/middleware/developer-tools/jdeveloper/14.1.2/user-guide/developing-applications-oracle-jdeveloper.pdf
31. The Day I Stopped Drinking Milk | PDF | Creative Commons License https://www.scribd.com/document/413767898/103792868-the-Day-i-Stopped-Drinking-Milk
32. [PDF] An Automated Framework for Deep Research Task Construction ... https://arxiv.org/pdf/2601.09688
33. Fundamentals and Practical Implications of Agentic AI - arXiv https://arxiv.org/html/2505.19443v1
34. Benchmarking Faithfulness Loss in Long-Horizon Coding Agents https://arxiv.org/html/2603.17104v1
35. SIN-Bench: Tracing Native Evidence Chains in Long-Context ... - arXiv https://arxiv.org/html/2601.10108v1
36. LLMSQL: Upgrading WikiSQL for the LLM Era of Text-to-SQL - arXiv https://arxiv.org/html/2510.02350v1
37. [PDF] Deploying Foundation Model Powered Agent Services: A Survey https://arxiv.org/pdf/2412.13437?
38. [PDF] Fundamentals and Practical Implications of Agentic AI - arXiv https://arxiv.org/pdf/2505.19443
39. [PDF] Tool-Aware Planning in Contact Center AI: Evaluating LLMs ... - arXiv https://www.arxiv.org/pdf/2602.14955
40. Reliable Graph-RAG for Codebases: AST-Derived Graphs vs LLM ... https://arxiv.org/html/2601.08773v1

41. Pre-Inference Prompt Mediation for Suppressing Privacy ... - arXiv https://arxiv.org/html/2604.05793v1
42. QEDBench: Quantifying the Alignment Gap in Automated Evaluation ... https://arxiv.org/html/2602.20629v1
43. [PDF] AST-Derived Graphs vs LLM-Extracted Knowledge Graphs - arXiv https://arxiv.org/pdf/2601.08773
44. [PDF] Pre-Inference Prompt Mediation for Suppressing Privacy ... - arXiv https://arxiv.org/pdf/2604.05793
45. [PDF] Benchmarking Faithfulness Loss in Long-Horizon Coding Agents https://arxiv.org/pdf/2603.17104
46. Context Engineering: A Methodology for Structured Human-AI ... https://arxiv.org/html/2604.04258v1
47. Infinity Synthetic Environments for Agentic Reinforcement Learning https://arxiv.org/html/2602.10090v2
48. The Long-Horizon Task Mirage? Diagnosing Where and Why ... - arXiv https://arxiv.org/html/2604.11978v1
49. On the Ability of Transformers to Verify Plans - arXiv https://arxiv.org/html/2603.19954v1
50. Scaling-Aware Adapter for Structure-Grounded LLM Reasoning - arXiv https://arxiv.org/html/2602.02780v1
51. τ-Knowledge: Evaluating Conversational Agents over Unstructured ... https://arxiv.org/html/2603.04370v1
52. JanusCoder: Towards a Foundational Visual-Programmatic ... - arXiv https://arxiv.org/html/2510.23538v2
53. [PDF] A Program-Structure Taxonomy of Smart Contract Vulnerabilities https://arxiv.org/pdf/2511.09051
54. The Rise of Manus AI as a Fully Autonomous Digital Agent - arXiv https://arxiv.org/html/2505.02024v2
55. Raiven: LLM-Based Visualization Authoring via Domain-Specific ... https://arxiv.org/html/2604.10008v1
56. Hallucination Detection in Foundation Models for Decision-Making https://arxiv.org/html/2403.16527v2
57. [PDF] An LLM Agent-based Framework for Whaling Countermeasures - arXiv https://arxiv.org/pdf/2601.14606
58. [PDF] Stock Market Prediction Using Node Transformer Architecture ... - arXiv https://arxiv.org/pdf/2603.05917

59. StarWhisper Telescope: Agent-Based Observation Assistant System ... https://arxiv.org/html/2412.06412v2
60. [PDF] Uvarov Alexey Viktorovich Variational quantum algorithms for local ... https://arxiv.org/pdf/2208.11220
61. [PDF] Dive into Deep Learning - arXiv https://arxiv.org/pdf/2106.11342
62. Computer Science - arXiv https://www.arxiv.org/list/cs/new?skip=275&show=500